

جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات

أنظمة التشغيل للأنظمة الذكية (ENGS2306)

المحاضرة: 7 برمجة Bash المتقدمة

المحاضر: المهندس محمد الهمص

المحاضرة 7: برمجة Bash المتقدمة

أهداف اليوم:

- 1. تنظيم الكود إلى وظائف قابلة لإعادة الاستخدام.
- 2. إدارة قوائم البيانات باستخدام المصفوفات.
- 3. تنفيذ معالجة قوية للأخطاء لجعل البرامج النصية أكثر موثوقية.
- 4. جمع هذه المفاهيم لكتابة نص برمجي معياري بمستوى احترافي.

ملخص: حدود النصوص الأساسية

في المحاضرة السادسة، تعلمنا أساسيات البرمجة النصية:

- المتغيرات لتخزين البيانات.
- إذا كانت البيانات تتعلق بالقرارات.
- حلقات while و for للتكرار.

المشكلة: مع ازدياد تعقيد مهامنا، قد تصبح نصوصنا البرمجية طويلة، يصعب قراءتها، وصيانتها. ماذا لو احتجنا إلى تنفيذ نفس المهمة المكونة من عشرة أسطر عدة مرات في نص برمجي واحد؟ نسخ ولصق الشيفرة البرمجية غير فعال، وهو سبب رئيسي للأخطاء البرمجية.

الحل: علينا أن نجعل نصوصنا البرمجية معيارية وقابلة للتطوير ومتينة. اليوم، سنتعلم كيفية القيام بذلك.

الكود المعياري مع الوظائف

الوظيفة عبارة عن كتلة مسماة من التعليمات البرمجية التي يمكنك استدعاؤها عدة مرات من أجزاء مختلفة من البرنامج النصي الخاص بك. وهذا يتبع مبدأ DRY (لا تكرر نفسك).

بناء الجملة الأساسي:

```
function_name() { #
    وظيفة الأولى!
}
```

```
--- #تشغيل الوظيفة، فقط اتصل باسمها function_name ---
```

لماذا استخدام الوظائف؟

- قابلية القراءة: يصبح الجزء الرئيسي من البرنامج النصي الخاص بك عبارة عن تسلسل واضح من استدعاءات الوظائف.
- إمكانية إعادة الاستخدام: اكتب المنطق مرة واحدة، واستخدمه في كل مكان.
- إمكانية الصيانة: إذا كنت بحاجة إلى إصلاح خطأ ما، يمكنك إصلاحه في مكان واحد فقط - الوظيفة.

الوظائف ذات المعلمات

يمكنك تمرير البيانات إلى دالة باستخدام معلمات موضعية. داخل الدالة، يتم الوصول إليها كـ \$1 (المتغير الأول)، و\$2 (المتغير الثاني)، وهكذا.

مثال:

```
#!/bin/bash # وظيفة تقوم بالترحيب بمستخدم معين.
```

```
greet_user() { local user_name=$1 # استخدام 'local' للمتغيرات داخل الدوال
echo "Hello, $user_name! Welcome."
```

```
}
```

```
# استدعاء الدالة بمعلمات مختلفة
greet_user "Ahmed" greet_user "Fatima"
```

@ \$ هو متغير خاص يمثل جميع الوسائط التي تم تمريرها إلى الدالة.

الحصول على البيانات من الوظائف

لا تُرجع دوال Bash البيانات كما تفعل لغات مثل Python أو C. لديها طريقتان رئيسيتان لتقديم النتائج:

١. حالة الخروج (للنجاح/الفشل): تُرجع الدالة رمز خروج (٠ للنجاح، ٢٥٥- للفشل). يمكنك التحقق من ذلك باستخدام # return \$? . نجاح

العودة # 1 فشل

٢. المخرجات القياسية (للبيانات): لإرجاع قيمة أو سلسلة نصية، تُرسل البيانات. يلتقط جزء الاستدعاء من البرنامج النصي هذه المخرجات باستخدام استبدال الأوامر.

مثال:

```
get_timestamp() { التاريخ "+%Y-%m-%d %H:%M:%S"
```

```
}
```

```
# التقاط الناتج المنعكس في متغير
log_time=$(get_timestamp) echo "حدث الحدث في: $log_time"
```

إدارة القوائم باستخدام المصفوفات

المصفوفة متغير خاص يمكنه استيعاب قائمة من قيم متعددة. وهو مثالي لإدارة قوائم الأجهزة أو المستخدمين أو الملفات .

بناء الجملة:

```
iot_devices=("192.168.1.50" "192.168.1.51" "sensor-gateway")
```

```
#{الوصول إلى عنصر واحد (يبدأ الفهرس عند echo "0"الجهاز الأول): "${iot_devices[0]}"
```

```
#{الوصول إلى جميع العناصر echo "جميع الأجهزة: "${iot_devices[@]}"
```

```
#{احصل على عدد العناصر echo "إجمالي الأجهزة: "${#iot_devices[@]}"
```

هام: عند توسيع المصفوفة، احرص دائمًا على وضع علامة اقتباس ("my_array[@]") للتعامل بشكل صحيح مع العناصر التي تحتوي على مسافات.

التكرار عبر المصفوفات

الاستخدام الأكثر شيوعًا للمصفوفة هو التكرار من خلالها باستخدام حلقة for.

بناء الجملة:

```
مصفوفتي =( "العنصر 1" "العنصر 2" "العنصر 3"
```

```
بالنسبة للعنصر الموجود في "${my_array[@]}": "قم بتنفيذ الأمر "جارٍ المعالجة: $item"
```

مثال على الأنظمة الذكية: برنامج نصي لتحديث البرامج على أجهزة Raspberry Pi المتعددة.

```
#!/bin/bash PI_HOSTS=("pi-zero1" "pi-compute4" "pi-gateway")
```

```
للمضيف في "${PI_HOSTS[@]}": "قم بإجراء echo "تحديث ---" ssh "pi@$host" "sudo apt update && sudo apt upgrade -y" echo "تم تحديث ---" $host انتهى "تم
```

معالجة قوية للأخطاء باستخدام set -e

ماذا يحدث إذا فشل أمر في البرنامج النصي؟ افتراضياً، يستمر البرنامج النصي في العمل، مما قد يؤدي إلى نتائج كارثية (مثل محاولة الكتابة إلى مجلد احتياطي لم يتم إنشاؤه).

set -e هو أمر يخبر shell بالخروج فوراً إذا فشل أي أمر (يرجع رمز خروج غير صفري).

ضع هذا بالقرب من أعلى البرامج النصية الخاصة بك لجعلها أكثر أماناً:

```
#!/bin/bash set -e
```

```
# سيفشل هذا الأمر إذا كان الدليل موجوداً بالفعل بدون # mkdir /nonexistent_root_dir/my_app # بسبب 'set -e' سيتوقف البرنامج النصي هنا. echo "لن يتم الوصول إلى هذا السطر أبداً."
```

هذه طريقة بسيطة ولكنها فعالة لضمان توقف التشغيل الآلي عند حدوث خطأ ما.

السيناريو العملي: صحة خدمة النظام يفحص

دعنا نكتب نصاً برمجياً يستخدم الوظائف والمصفوفات للتحقق من حالة الخدمات الهامة للنظام.

1. قم بإنشاء ملف البرنامج النصي: touch health_check.sh && chmod +x health_check.sh

2. أضف الأساسيات: set -e. و#!/bin/bash

3. قم بتحديد مجموعة من الخدمات للتحقق منها: SERVICES=("ssh" "cron" "ufw")

4. قم بإنشاء دالة check_service() تأخذ وسيطة واحدة (اسم الخدمة).

5. داخل الدالة، استخدم "\$1" --quiet systemctl is-active للتحقق من الخدمة. علامة --quiet

يجعلها تعيد رمز الخروج فقط 0 إذا كانت نشطة، وغير صفرية إذا كانت غير نشطة/فشلت).

6. داخل الدالة، استخدم عبارة if للتحقق من رمز الخروج (\$?) الأمر systemctl.

اطبع رسالة النجاح أو الفشل.

7. في الجزء الرئيسي من البرنامج النصي، اطبع رأساً مثل "--- System Health Report ---"

8. استخدم حلقة for للتكرار خلال مجموعة الخدمات الخاصة بك.

9. داخل الحلقة، قم باستدعاء وظيفة check_service() لكل خدمة.

10. قم بتشغيل البرنامج النصي: ./health_check.sh

الفكرة وراء المهمة

لماذا تعتبر هذه المفاهيم المتقدمة مهمة للأنظمة الذكية؟

• التركيبية والتعقيد: النظام الذكي الحديث (مثل الروبوت أو مركز المنزل الذكي) ليس برنامجًا واحدًا؛ بل هو عشرات الخدمات التي تعمل معًا. يجب أن يكون نص الإدارة تركيبياً. يمكنك استخدام وظائف مثل `read_all_sensors()`، `grestart_wifi()` و `check_camera_feed()`

يظل منطق النص الرئيسي واضحًا ومفهوماً.

• قابلية التوسع: تخيل نشر أسطول من 100 مستشعر إنترنت الأشياء. لا يتطلب الأمر كتابة 100 نص برمجي مختلف. تكتب نصًا برمجيًا واحدًا يقرأ قائمة عناوين IP لأجهزة الاستشعار من مجموعة (أو ملف) ويمر عبرها، وينفذ نفس مهمة التكوين أو التحديث على كل منها.

الموثوقية والسلامة: بالنسبة للنظام الذي يعمل بشكل مستقل (مثل طائرة بدون طيار أو وحدة تحكم صناعية)، فإن الفشل غير المُراقب ليس خيارًا. تضمن معالجة الأخطاء بدقة (set -e) والتحقق من رموز الخروج) أنه في حال تعطل أحد المكونات الأساسية (مثل عدم استجابة وحدة تحكم المحرك)، يتوقف البرنامج النصي بأمان بدلاً من الاستمرار في عملية خطيرة أو فاشلة جزئيًا.

المهمة 6:مُراسل صحة النظام المعياري (12%)

الهدف: كتابة نص برمجي معياري وسهل الاستخدام باستخدام الوظائف والمصفوفات والمنطق الشرطي لإنشاء تقرير عن صحة النظام.

المهام:

1.قم بإنشاء البرنامج النصي باسم `system_reporter.sh`.

2.في الأعلى، قم بتضمين `#!/bin/bash` و `set -e`.

3.قم بتحديد مجموعة تحتوي على أسماء ثلاث خدمات على الأقل تريد التحقق منها (على سبيل المثال، `ssh`، `cron`، `ufw`، `cups`).

4.أنشئ دالة باسم `check_service_status()` ستأخذ هذه الدالة وسيطة واحدة: الاسم من الخدمة.

5.داخل الوظيفة، يجب التحقق مما إذا كانت الخدمة نشطة باستخدام `systemctl is-active --quiet "$1"`.

6.بناءً على رمز الخروج (`?`) يجب أن تطبع الدالة الحالة بتنسيق ملون وسهل الاستخدام. على سبيل المثال:

•إذا كان نشطًا: `echo -e "[موافق] $1 نشط."`

•إذا كان غير نشط/فشل: `echo -e "[FAIL] $1 ليس نشطًا."`

7.في الجزء الرئيسي من النص الخاص بك:

•اطبع رأس الصفحة، على سبيل المثال، "تقرير صحة النظام لـ `$(hostname)` اعتبارًا من `$(date)`"

• استخدم حلقة for للتكرار خلال مجموعة الخدمات الخاصة بك.

• داخل الحلقة، قم باستدعاء دالة check_service_status لكل خدمة.

• طباعة تذييل، على سبيل المثال، ---"نهاية التقرير".---

تعليمات تسليم الواجبات

• اسم الملف: قم بتسمية ملف الإرسال الخاص بك بالضبط كما يلي:

رقم الطالب-اسم الطالب-الواجب 6.pdf

• مثال: 13012070xx-MohammedEmadAlhams-Assignment6.pdf

• موجه المحطة الطرفية: يجب أن يظهر اسمك في موجه المحطة الطرفية للقطات الشاشة الخاصة بك.

• تنسيق الملف: يجب أن يكون إرسالكم عبارة عن ملف PDF واحد يحتوي على:

• الكود الكامل لملف system_reporter.sh الخاص بك (نسخ ولصق كنص).

• لقطة شاشة تظهر مخرجات البرنامج النصي الذي تم تشغيله بنجاح.

• تقرير موجز (فقرة أو فقرتين) يشرح الفائدة الأساسية لاستخدام الوظيفة

(check_service_status) في البرنامج النصي الخاص بك مقارنة بوضع كل منطق التحقق مباشرة داخل حلقة for.

• النزاهة الأكاديمية:

• تحذير: سيتم اعتبار المشاركات المتطابقة غشًا وستؤدي إلى الحصول على درجة صفر.

• تحذير: الحلول أو التقارير التي تم إنشاؤها بواسطة الذكاء الاصطناعي ستؤدي إلى الحصول على درجة صفر.

الخطوات التالية والموارد

تمرين هذا الاسبوع:

• عدّل مُبلِّغ الصحة لديك. هل يمكنك إنشاء دالة أخرى للتحقق من مساحة القرص (df -h)؟

اضافته الى التقرير؟

• حاول تمرير وسيطتين إلى دالة. على سبيل المثال، دالة تُنشئ دليلاً (1 دولار) وتُحدد أذونات (2 دولار).

الأسبوع القادم: المحاضرة - 8 أساسيات شبكات لينكس

سننتقل من البرمجة النصية إلى الشبكات. سنتعلم الأوامر الأساسية لتكوين اتصالات الشبكة وإدارتها واستكشاف أخطائها وإصلاحها على لينكس، وهي مهارة أساسية لأي نظام ذكي متصل.

نصائح الاتصال بغزة:

• حمل هذه المحاضرة وتدرّب عليها دون اتصال بالإنترنت. لا يحتاج جهازك الافتراضي إلى اتصال بالإنترنت لكتابة نصوص Bash واختبارها.

• التركيز على فهم المنطق، وليس مجرد نسخ الكود.

موارد:

• وظائف Bash والمتغيرات المحلية: <https://tldp.org/LDP/abs/html/functions.html>

• مصفوفات Bash: <https://opensource.com/article/18/5/you-dont-know-bash-intro-bash-arrays>

• معالجة الأخطاء / https://vaneyckt.io/posts/safer_bash_scripts_with_set_euxo_pipefail/ (set -e):

• مواد الدورة على Moodle